Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 4

Exercise 4.1 – Textured sky sphere + grass floor (Reference)

Task 1 – Identify the scene setup

1. It is the Renderer object for rendering, Camera object for user pov, and MovementRig for handling movement.
2. MovementRig object
3. Renderer object

Task 2 – Camera & rig

1. Because we need to know which is the current position and the view of user. Then we might handle the background surroundings later.
2. As the xyz axis, the camera view will set on higher (y = 5) and change the position at z = 12.

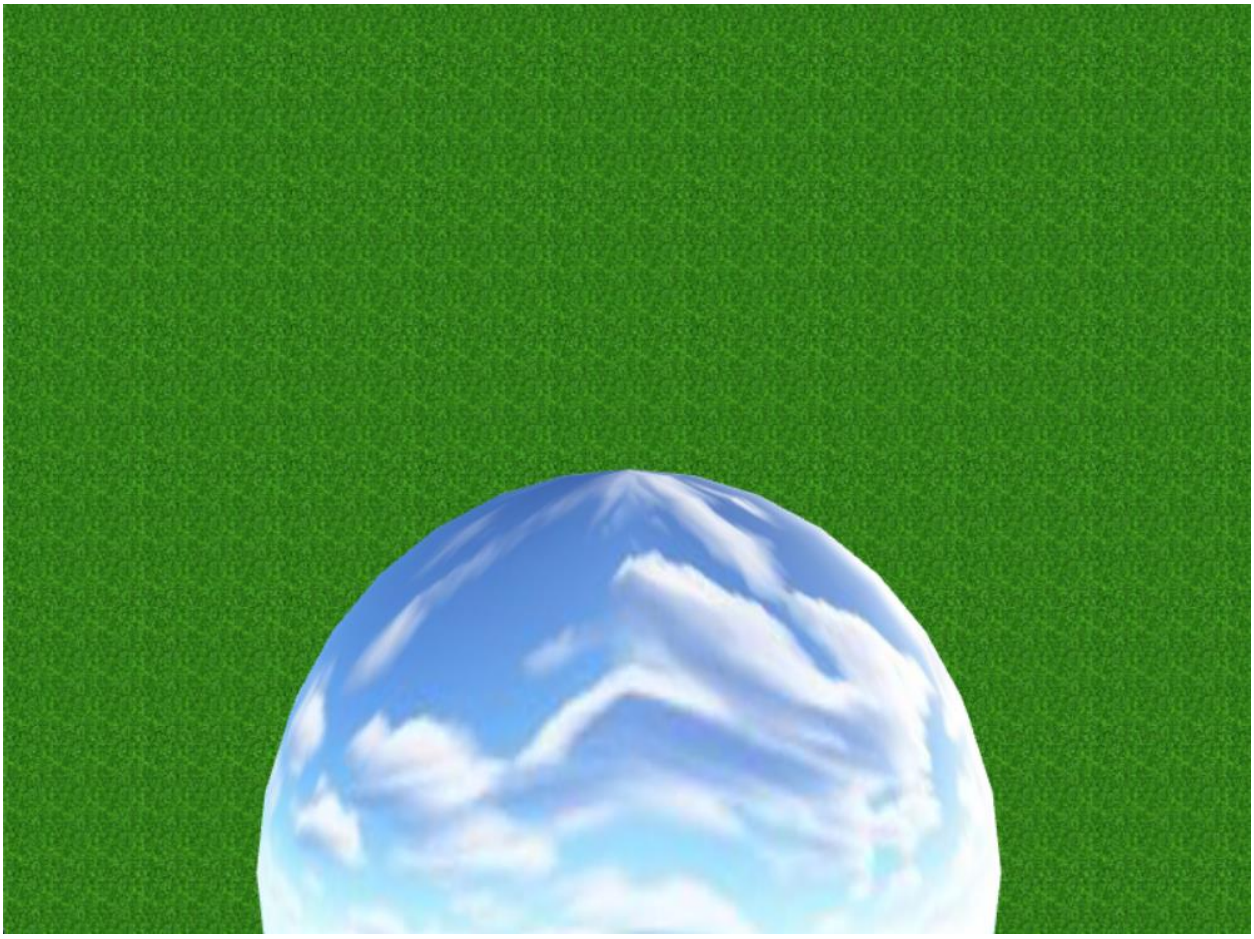
Task 3 – Sky sphere

1. The radius of the sky sphere is the radius of the sky that will surround the object, including the camera and grass inside. The reason for the large radius (50) is ensuring the sky can surround all the objects inside and make it looks fay away from the ground.
2. The sky will become smaller.

3. It can help optimize the rendering resource. If the sky will far away, it will handle all the location that we cannot see, which can make the application lagging and the device might not handle unnecessary rendered objects. So the sky around the camera can solve this issue.

Task 4 – Grass plane orientation

1. When we run, the grass and the sky will appear in front of us. So rotating it by -math.pi/2 make the grass rotates 90 degrees and becomes the ground.
2. It will appear in front of us, or 0 degrees on the screen.



Task 5 – Texture tiling

1. It will cut the texture, which is currently the grass texture tiling. 50, 50 meaning cutting 50 times into tiles, make the grass texture become smoother and the ground become larger.

2.

Yes. Because it renders and cuts the texture once.

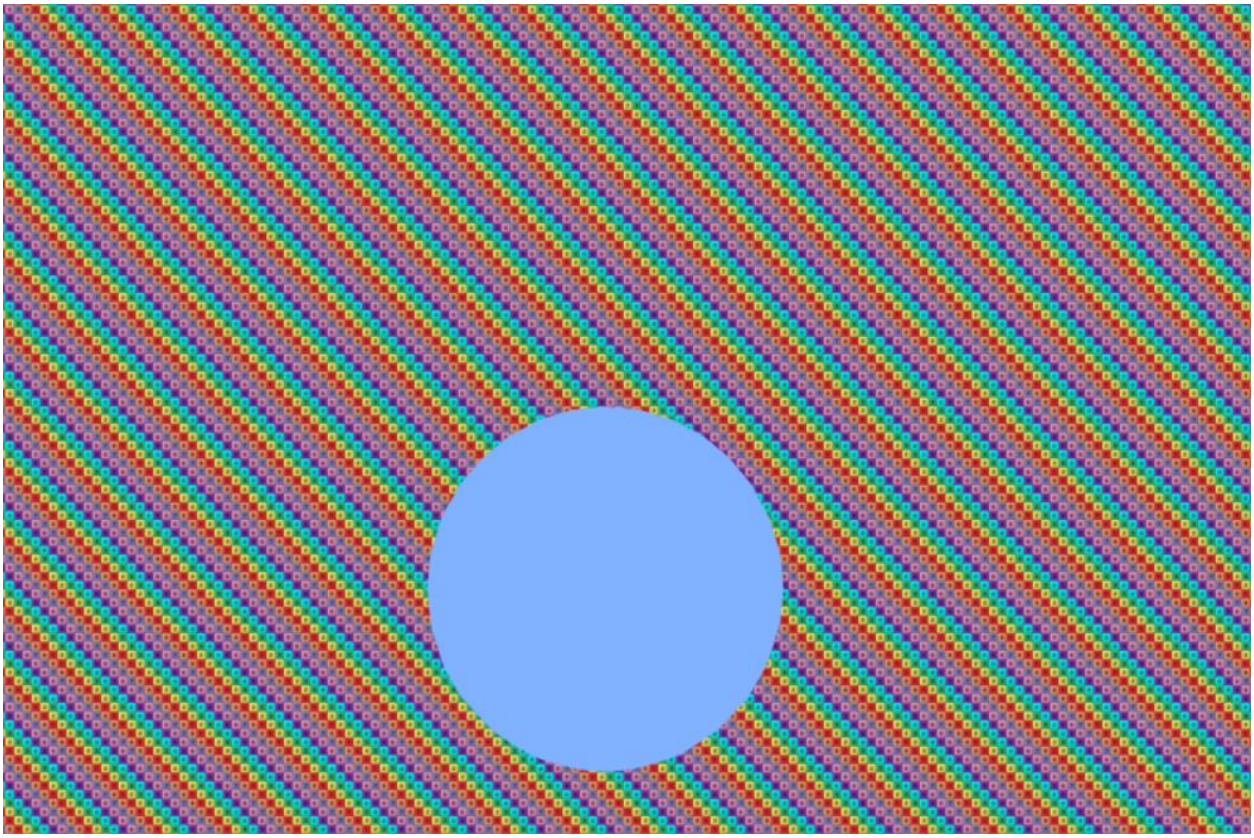
3. When we cut the texture into many tiles, it can make low texel density and make the ground smoother.

Task 6 – Render loop

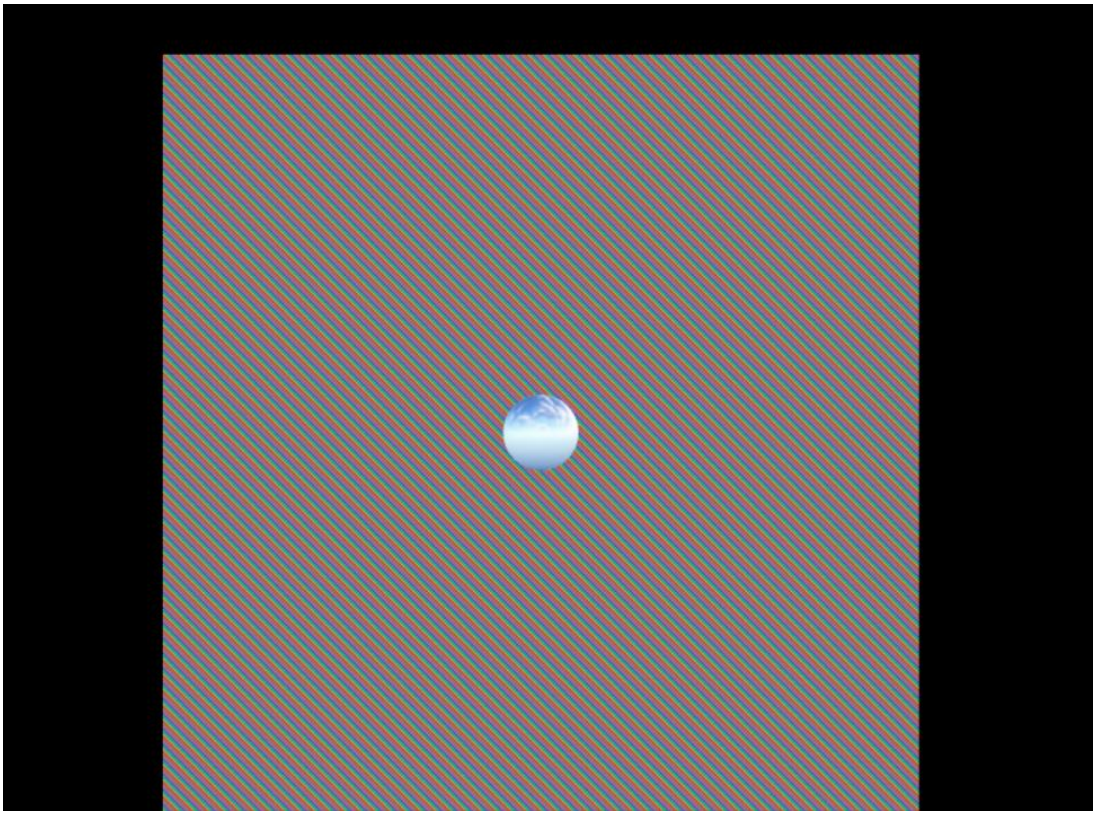
We call the self.rig.update(self.input, self.delta_time) for camera location updating, and self.renderer.render(self.scene, self.camera) for camera view updating.

1. If we comment out the first line, we cannot move.
2. If we move as fast enough, it will have some issue about render because we will render before we move. For example, some objects might not appear when we arrive.

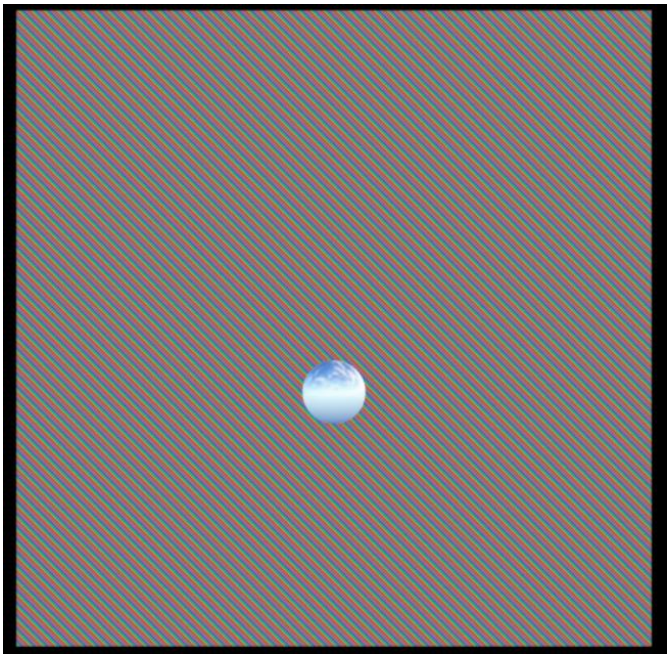
Task 7 – Extra mini-mods (quick checks)



Change the texture and sky texture to a solid-color texture.

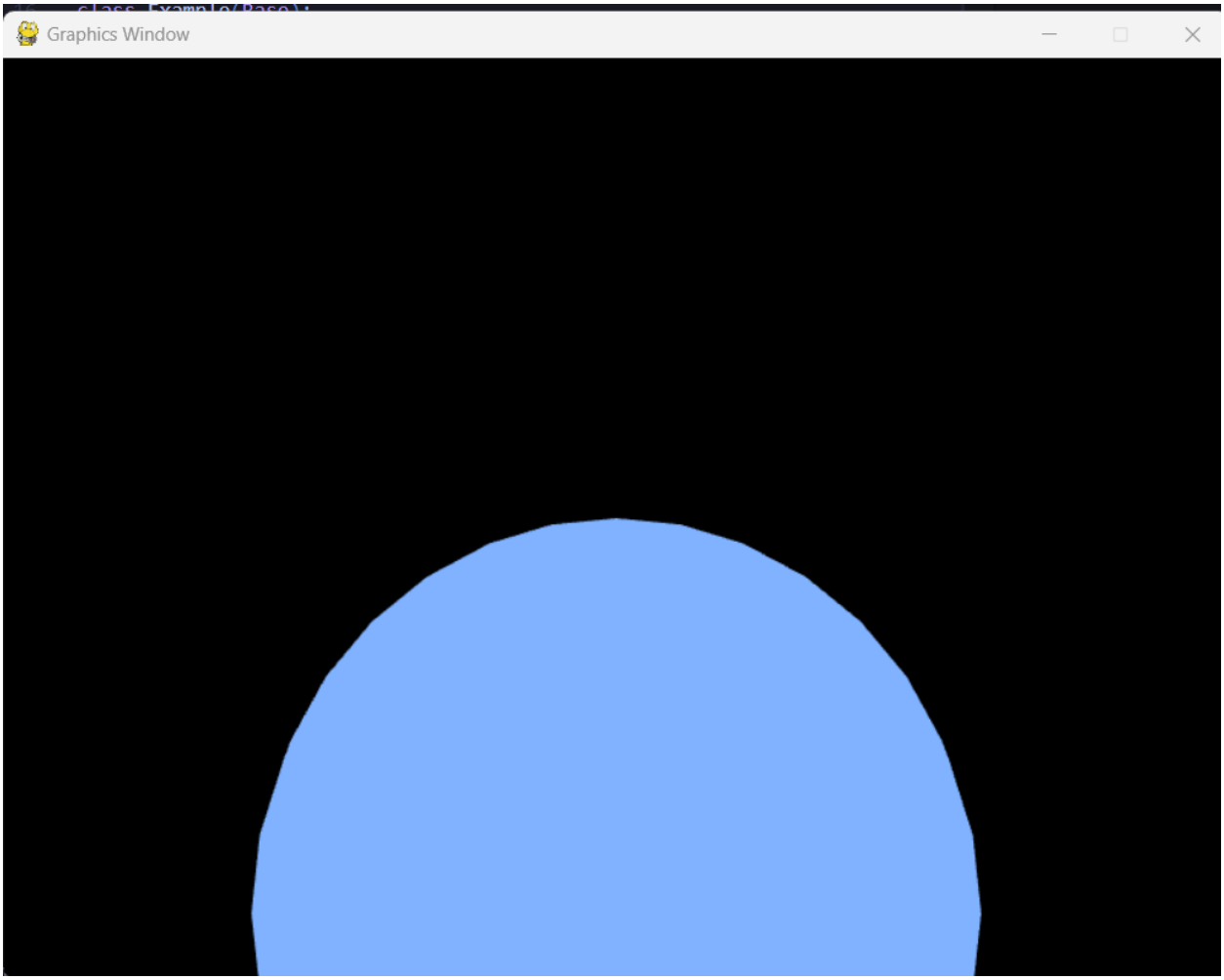


The grass.set_position([0, -0.1, 0]) move the ground down for a bit, as y = -0.1
If we set grass.set_position([0, 10, 0]), the ground will move up as below.



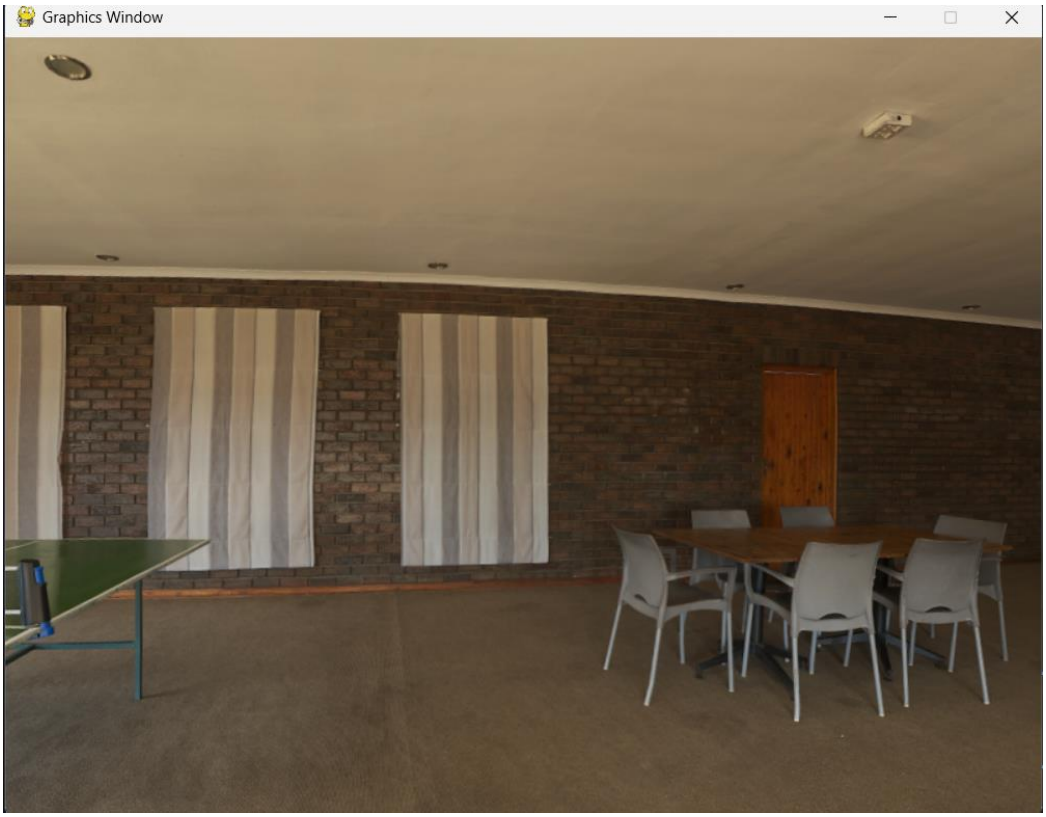
Exercise 4.2 – From sky+ground → indoor 360° room

Task 1 – Remove the ground



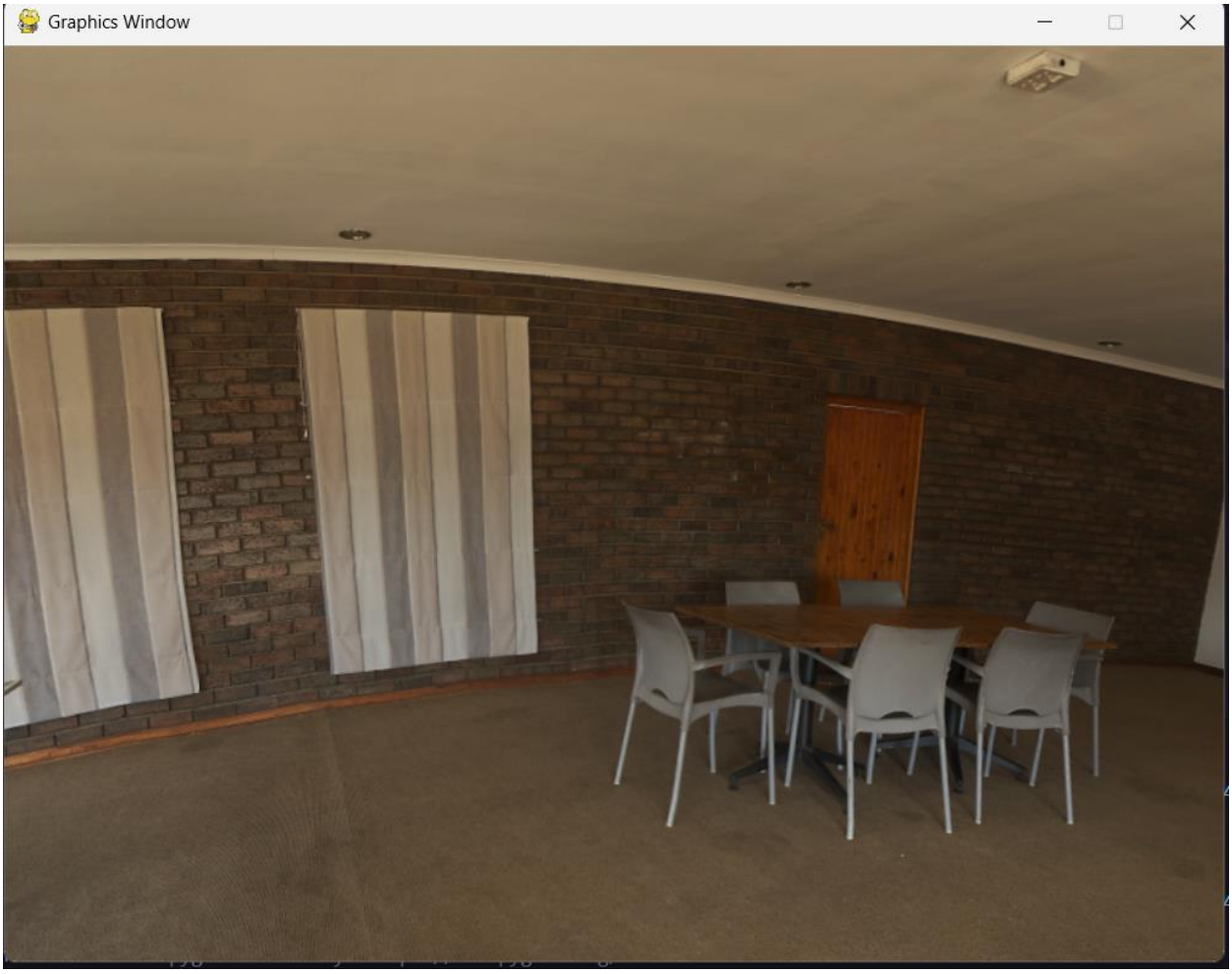
Task 2 – Change the texture to the indoor one

Task 3 – Make the camera stand in the center



Task 4 – Flip the sphere so we can see the inside

```
        sky_material = TextureMaterial(texture=Texture(file_name="textures/empty_play_room.jpg"))
        sky = Mesh(sky_geometry, sky_material)
        sky.scale = [-1, 1, 1]
        self.scene.add(sky)
```



Task 5 – Keep the sphere very large

We use a big radius, so the sky looks infinitely far and stays still around the viewer.

Task 6 – Increase movement speed (optional but recommended)

```
        self.camera = Camera(aspect_ratio=800/600)
        self.rig = MovementRig(units_per_second=100, degrees_per_second=60)
```

➔ The camera moves faster.

Task 7 – Answer

What is the one line that actually turns the sphere into an indoor environment?

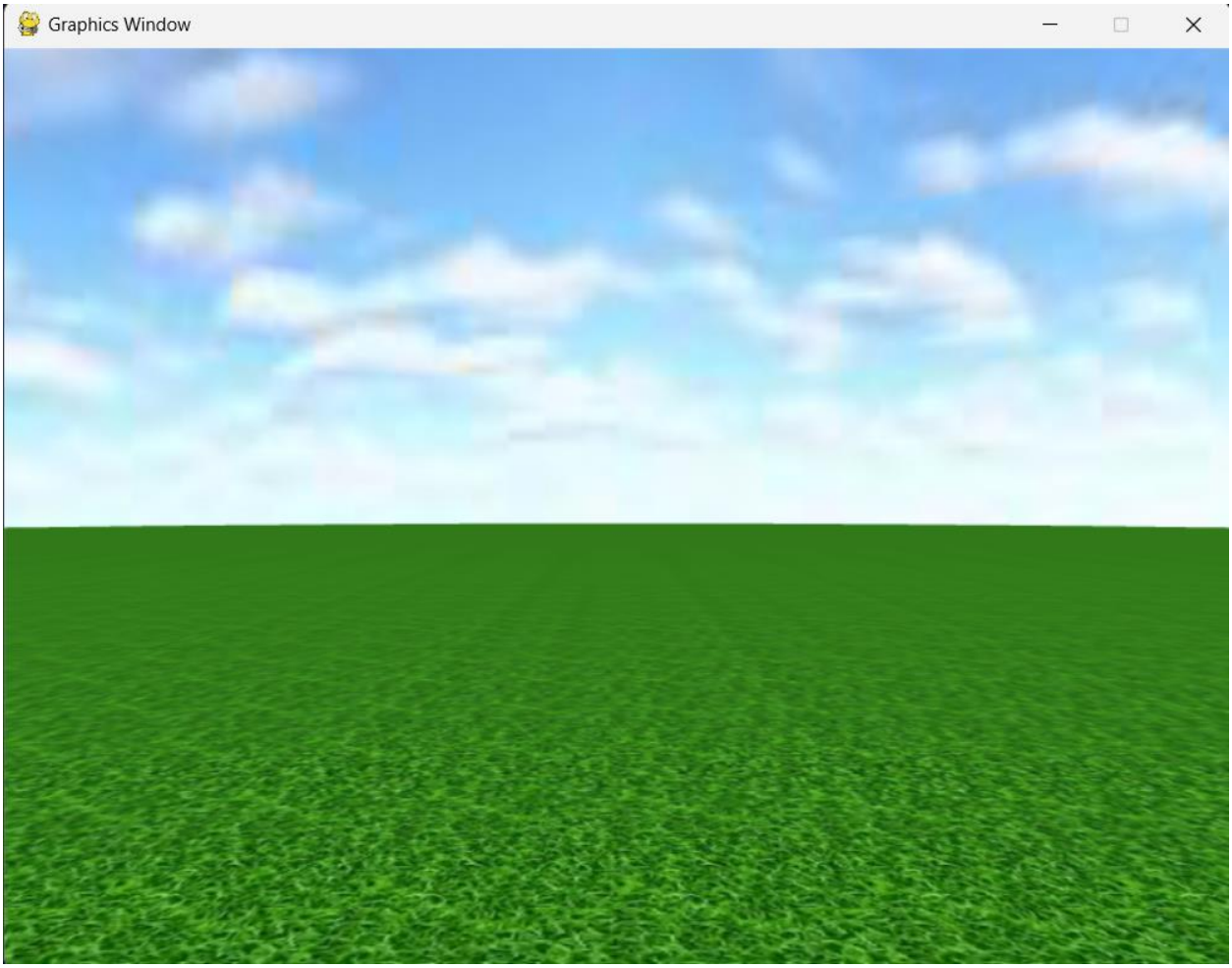
Ans: sky.scale = [-1, 1, 1]

What would be an alternative (besides scaling) to make an inside-visible sphere? (hint: flip faces)

Ans: sky_geometry.flip_faces()

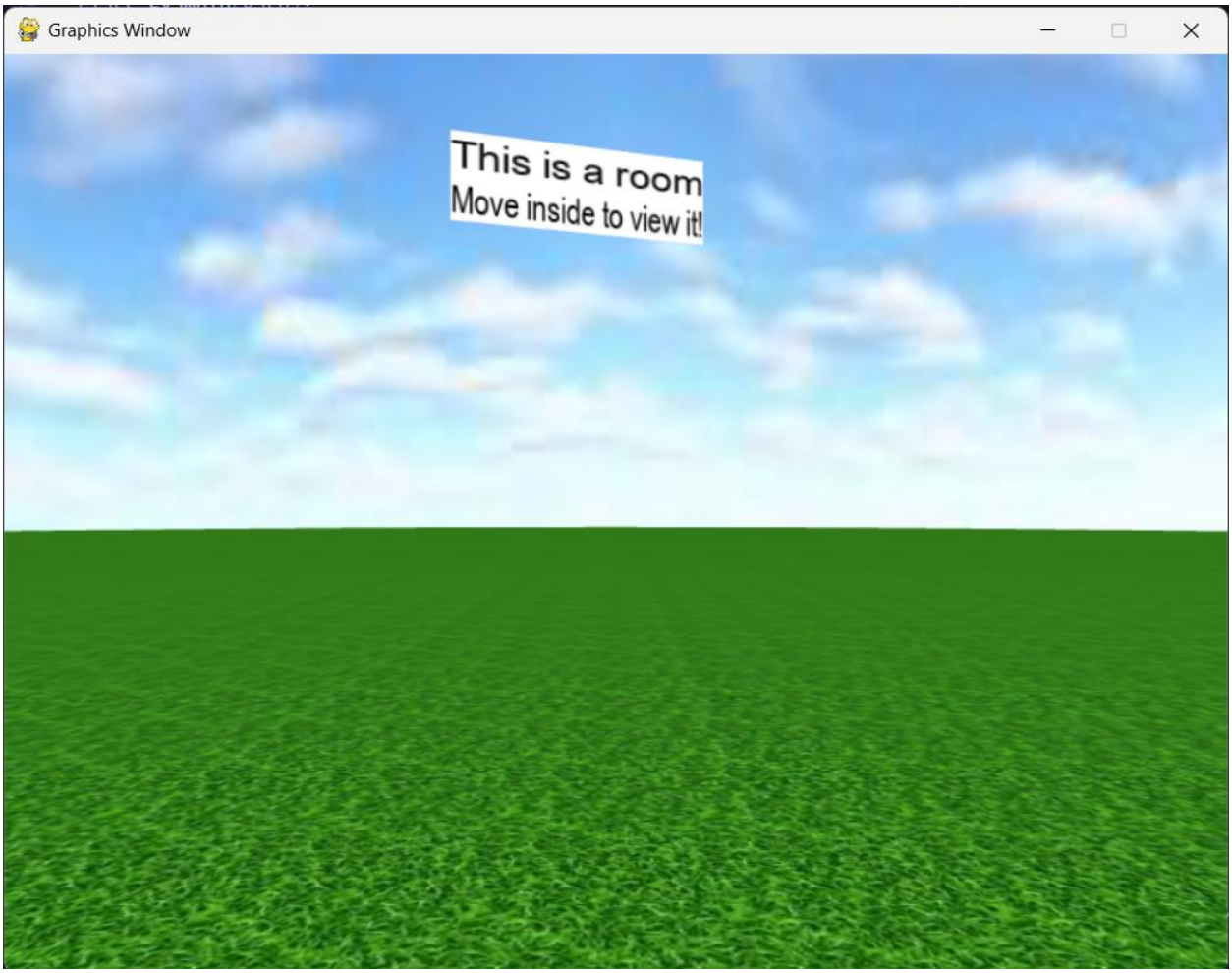
Exercise 4.3 – Two cameras + text textures inside a room Start from your 4.2 (indoor sphere) or 4.1 (sky+grass) and follow these steps.

Task 1 – Restore outdoor scene



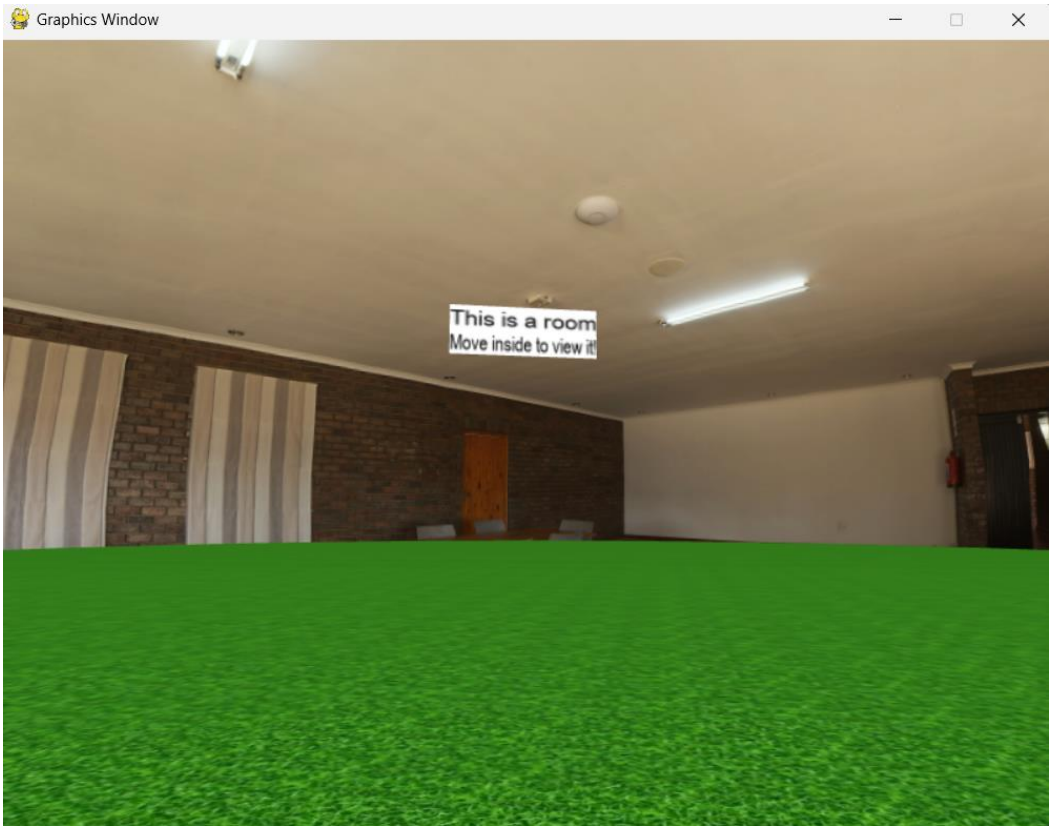
Task 2 – Add 2 text boards

```python
#!/usr/bin/python3
import math

from py3d.core.base import Base
from py3d.core_ext.camera import Camera
from py3d.core_ext.mesh import Mesh
from py3d.core_ext.renderer import Renderer
from py3d.core_ext.scene import Scene
from py3d.core_ext.texture import Texture
from py3d.geometry.rectangle import RectangleGeometry
from py3d.geometry.sphere import SphereGeometry
from py3d.material.texture import TextureMaterial
from py3d.extras.movement_rig import MovementRig
from py3d.extras.text_texture import TextTexture

class Example(Base):
    """
    Render a textured skysphere and a textured grass floor.
    Move the camera: WASDRF(move), QE(turn), TG(look).
    """
    def initialize(self):
        print("Initializing program...")
        self.renderer = Renderer()
        self.scene = Scene()
        self.camera = Camera(aspect_ratio=800/600)
        self.rig = MovementRig()
        self.rig.add(self.camera)
        self.scene.add(self.rig)
        self.rig.set_position([0, 1, 4])
        sky_geometry = SphereGeometry(radius=50)
        sky_material = TextureMaterial(texture=Texture(file_name="textures/sky.jpg"))
        sky = Mesh(sky_geometry, sky_material)
        self.scene.add(sky)
        grass_geometry = RectangleGeometry(width=100, height=100)
        grass_material = TextureMaterial(
            texture=Texture(file_name="textures/grass.jpg"),
            property_dict={"repeatUV": [50, 50]}
        )
        grass = Mesh(grass_geometry, grass_material)
        grass.rotate_x(-math.pi/2)
        self.scene.add(grass)
        text1 = TextTexture(text="This is a room")
        text1_material = TextureMaterial(texture=text1)
        text1_geo = RectangleGeometry(width=3.0, height=0.5)
        text1_mesh = Mesh(text1_geo, text1_material)
        text1_mesh.set_position([0, 5, 0])
        text1_mesh.rotate_y(math.pi)
        self.scene.add(text1_mesh)
        text2 = TextTexture(text="Move inside to view it!")
        text2_material = TextureMaterial(texture=text2)
        text2_geo = RectangleGeometry(width=3.0, height=0.5)
        text2_mesh = Mesh(text2_geo, text2_material)
        text2_mesh.set_position([0, 4.5, 0])
        text2_mesh.rotate_y(math.pi)
        self.scene.add(text2_mesh)
        self.text_objects = [text1_mesh, text2_mesh]

    def update(self):
        self.renderer.render(self.scene, self.camera)
        self.rig.update(self.input, self.delta_time)
        for t in self.text_objects:
            t.rotate_y(0.3 * self.delta_time)


# Instantiate this class and run the program
Example(screen_size=[800, 600]).run()
```



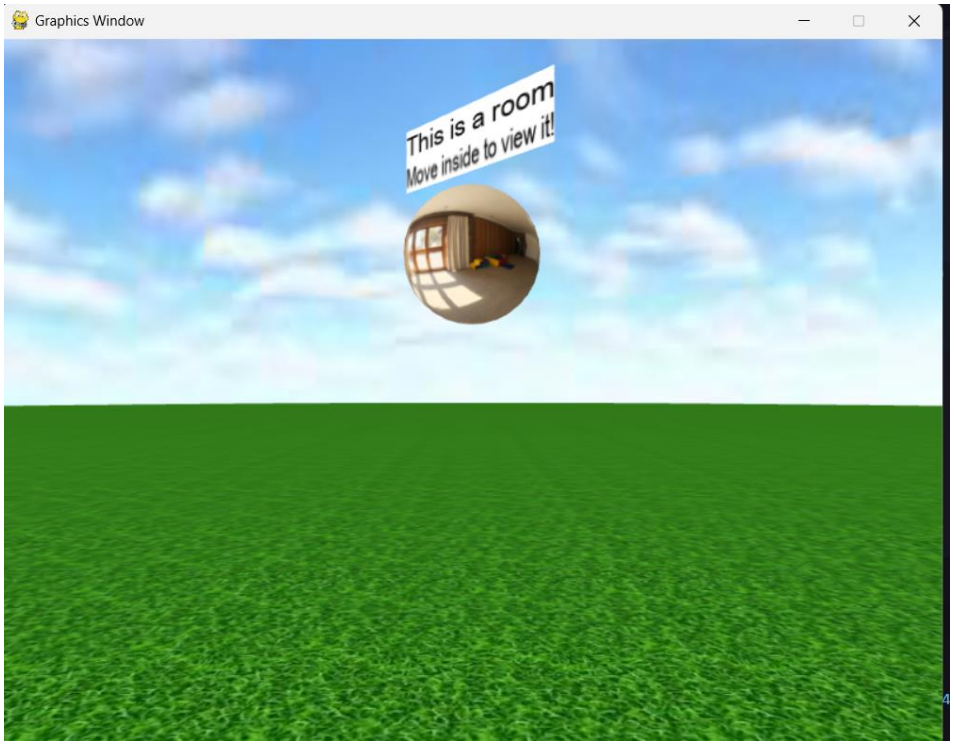
Task 3 – Add indoor environment sphere

```
        room_geometry = SphereGeometry(radius=20.0)
        room_material = TextureMaterial(texture=Texture(file_name="textures/empty_play_room.jpg"))
        room = Mesh(room_geometry, room_material)
        room.scale = [-1, 1, 1]
        room.set_position([0, 3, 0])
        self.scene.add(room)
```



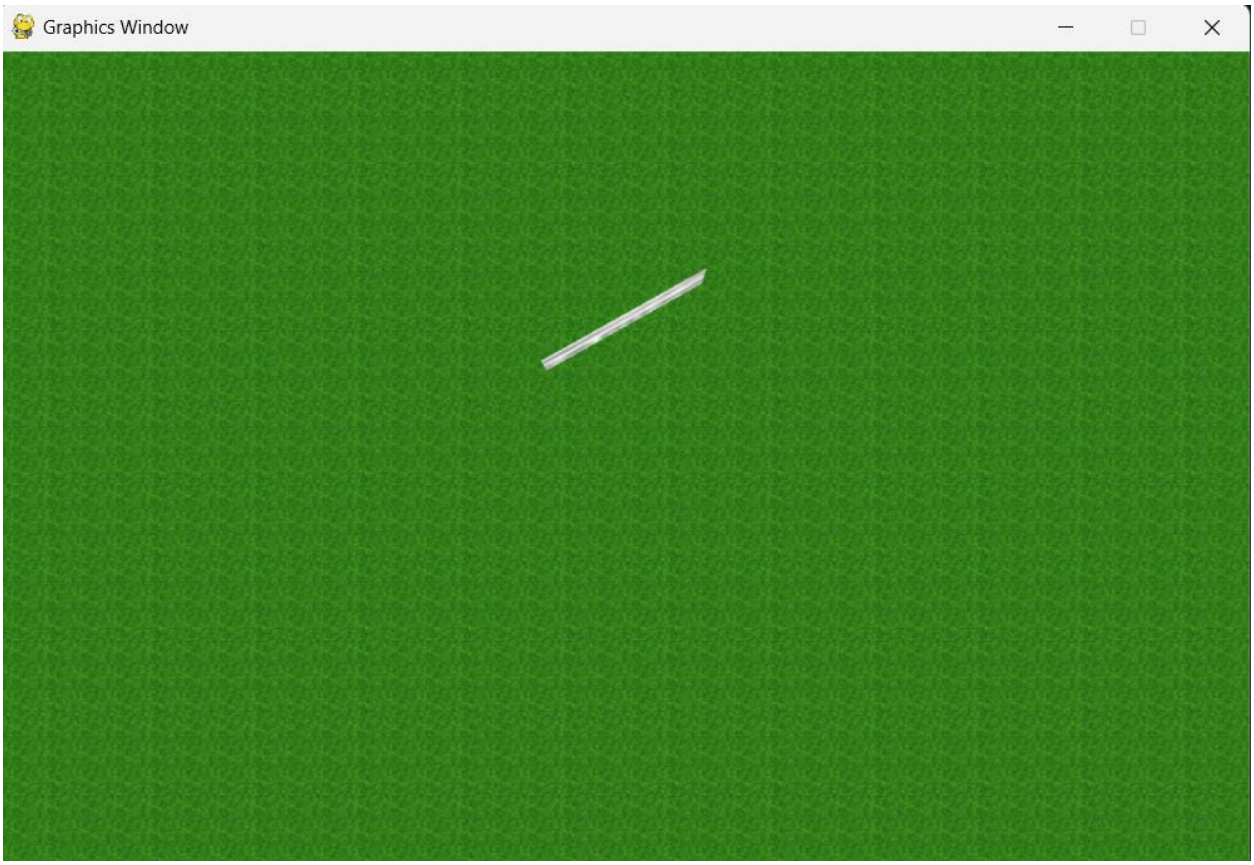
Why is a very small radius not good here?

Ans: Because small radius makes the camera outside the indoor sphere.



Task 4 – Add a second camera

```
26          self.sky_camera = Camera(aspect_ratio=512/512)
27          self.sky_camera.set_position([0, 20, 0])
28          self.sky_camera.look_at([0, 0, 0])
29          self.rig = MovementRig()
30          self.rig.add(self.camera)
31          self.rig.add(self.sky_camera)
```

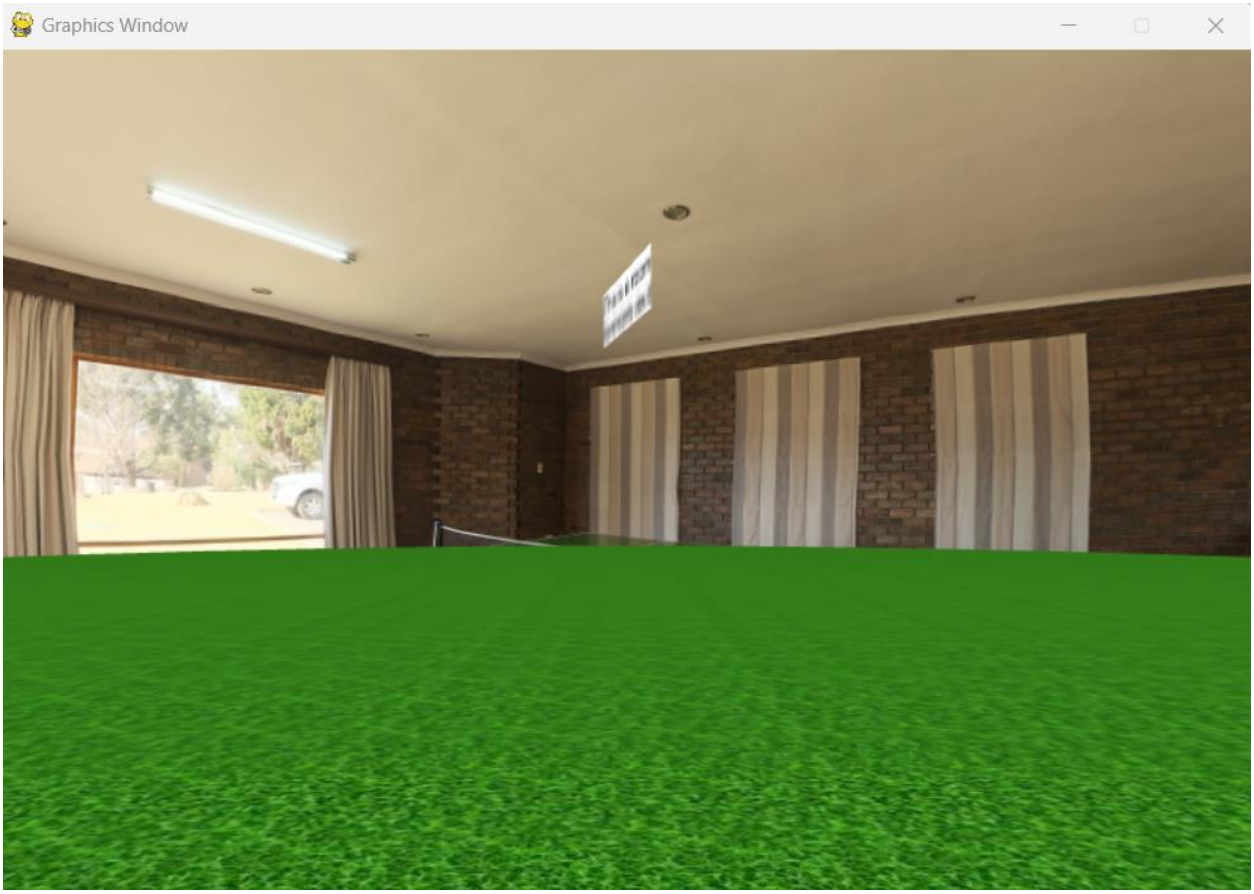


Why do we put it high on the Y-axis?

Ans: Normally we use it for mini-map, so we place the sky camera high on the Y-axis that it can view the entire scene from above, giving an overview of the environment.

Task 5 – Render with both cameras

```
68      def update(self):
69          for t in self.text_objects:
70              t.rotate_y(0.3 * self.delta_time)
71          self.rig.update(self.input, self.delta_time)
72          self.renderer.render(self.scene, self.sky_camera
73          self.renderer.render(self.scene, self.camera)
74
```



Why does the main camera “win”?

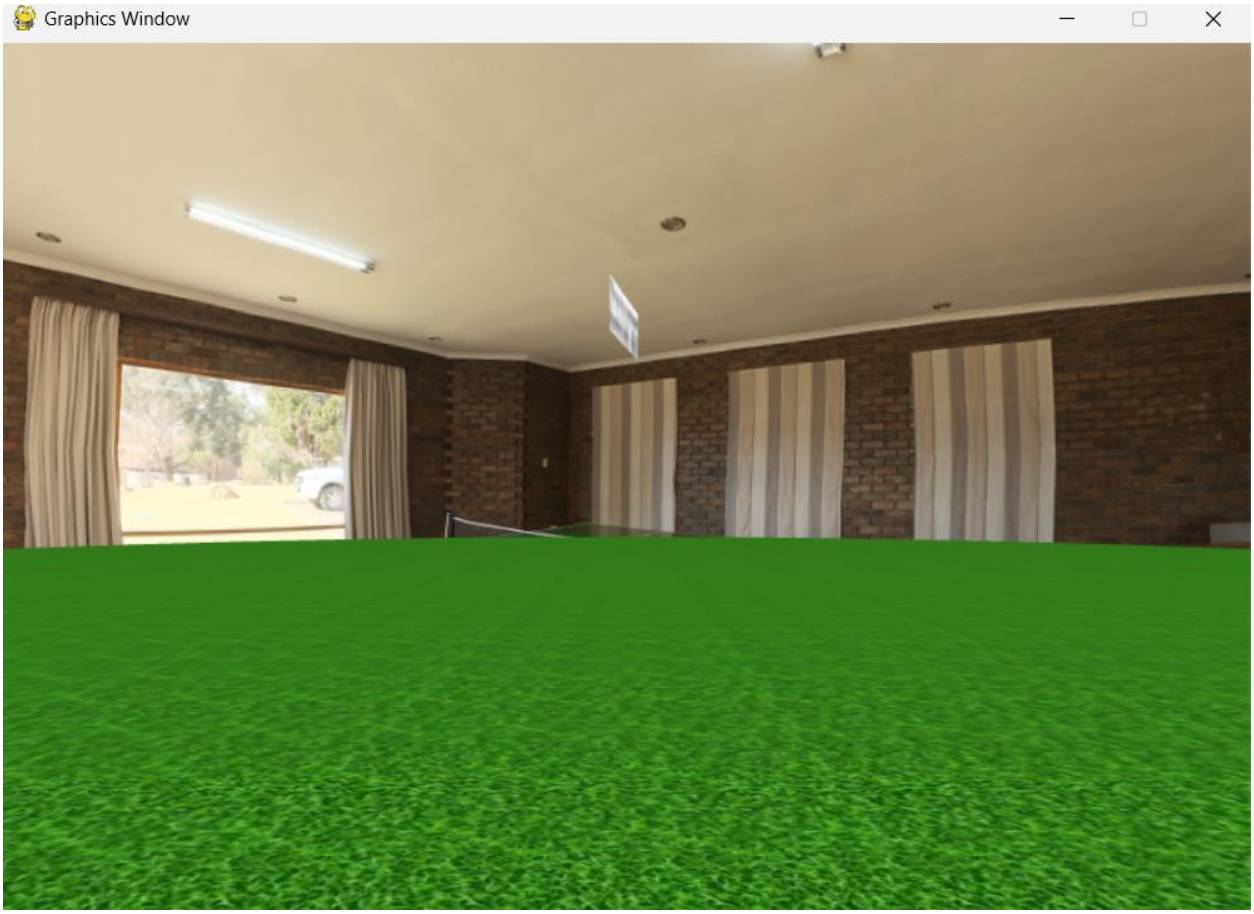
Ans: Since each render() call erases and overwrites the old frame, the last rendered camera is always the last one displayed on screen.

Task 6 – Explain why we need two cameras

Normally, we use the first main camera for interaction, movement. The second camera will be used as a mini-map, which can view the entire scene from above, giving an overview of the environment.

Exercise 4.4 – Build a room from 6 rectangles

Task 1 – Base scene again



Task 2 – Camera outside the room

```
self.rig = MovementRig(units_per_second=100, degrees_per_second=60)
self.rig.add(self.camera)
self.rig.add(self.sky_camera)
self.scene.add(self.rig)
self.rig.set_position([0, 1, 6])
```

Task 3 – Room size (important constants)

Task 4 – Floor

```
        floor_geometry = RectangleGeometry(width=room_w, height=room_d)
        floor_material = TextureMaterial(texture=Texture(file_name="textures/grid.jpg"))
        floor = Mesh(floor_geometry, floor_material)
        floor.rotate_x(-math.pi / 2)
        floor.set_position([0, 0, 0])
        self.scene.add(floor)
```

Task 5 – Ceiling

```
        ceiling_geometry = RectangleGeometry(width=room_w, height=room_d)
        ceiling_material = TextureMaterial(texture=Texture(file_name="textures/grid.jpg"))
        ceiling = Mesh(ceiling_geometry, ceiling_material)
        ceiling.rotate_x(math.pi / 2)
        ceiling.set_position([0, room_h, 0])
        self.scene.add(ceiling)
```

Task 6 – Back wall (−Z)

```
        back_geometry = RectangleGeometry(width=room_w, height=room_h)
        back_material = TextureMaterial(texture=Texture(file_name="textures/grid.jpg"))
        back = Mesh(back_geometry, back_material)
        back.set_position([0, room_h / 2, -room_d / 2])
        self.scene.add(back)
```

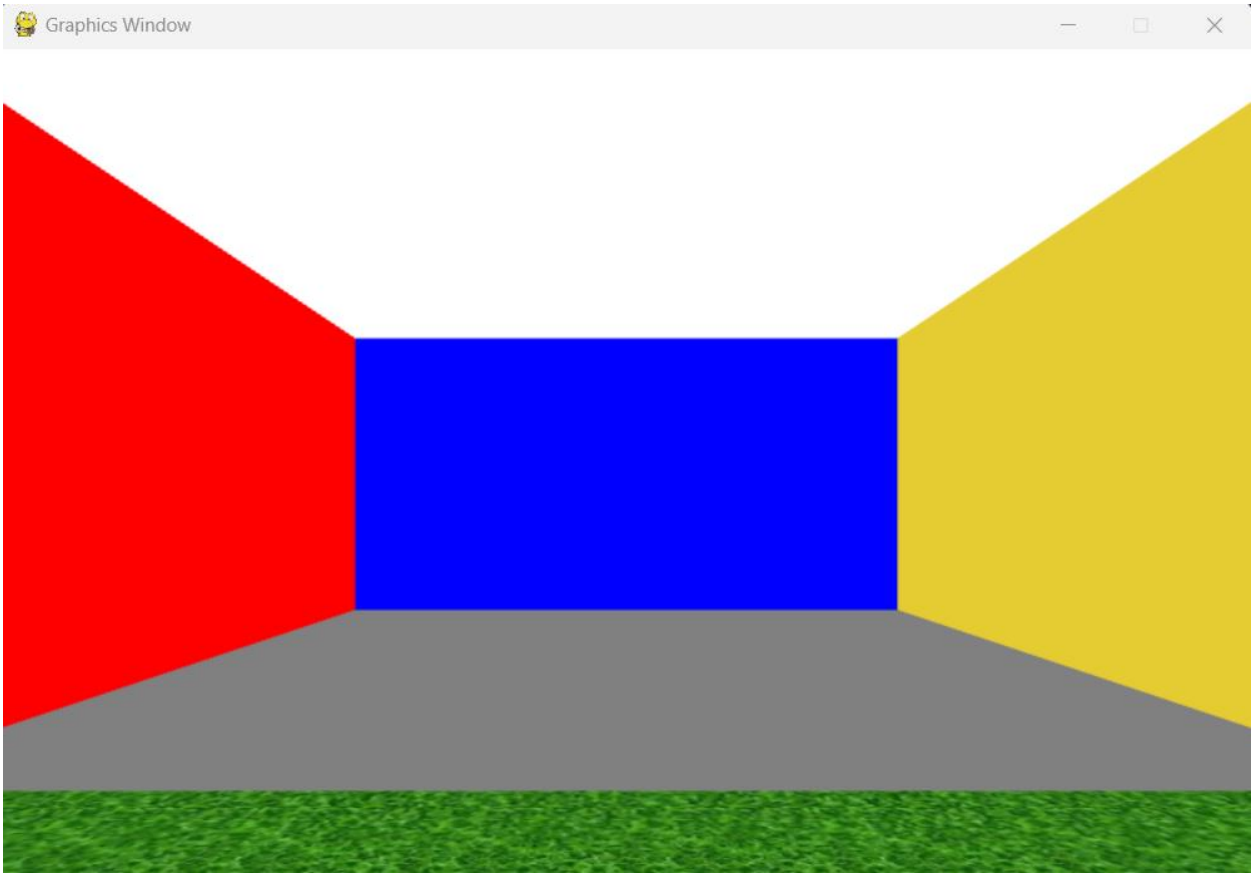
Task 7 – Front wall (+Z) facing inward

```
86          front_geometry = RectangleGeometry(width=room_w, height=room_h)
87          front_material = TextureMaterial(texture=Texture(file_name="textures/grid.jpg"))
88          front = Mesh(front_geometry, front_material)
89          front.rotate_y(math.pi)
90          front.set_position([0, room_h / 2, room_d / 2])
91          self.scene.add(front)
```

Task 8 – Left / right walls

```
92          left_geometry = RectangleGeometry(width=room_d, height=room_h)
93          left_material = TextureMaterial(texture=Texture(file_name="textures/grid.jpg"))
94          left = Mesh(left_geometry, left_material)
95          left.rotate_y(math.pi / 2)
96          left.set_position([-room_w / 2, room_h / 2, 0])
97          self.scene.add(left)
98          right_geometry = RectangleGeometry(width=room_d, height=room_h)
99          right_material = TextureMaterial(texture=Texture(file_name="textures/grid.jpg"))
100         right = Mesh(right_geometry, right_material)
101         right.rotate_y(-math.pi / 2)
102         right.set_position([room_w / 2, room_h / 2, 0])
103         self.scene.add(right)
```

Task 9 – Text boards



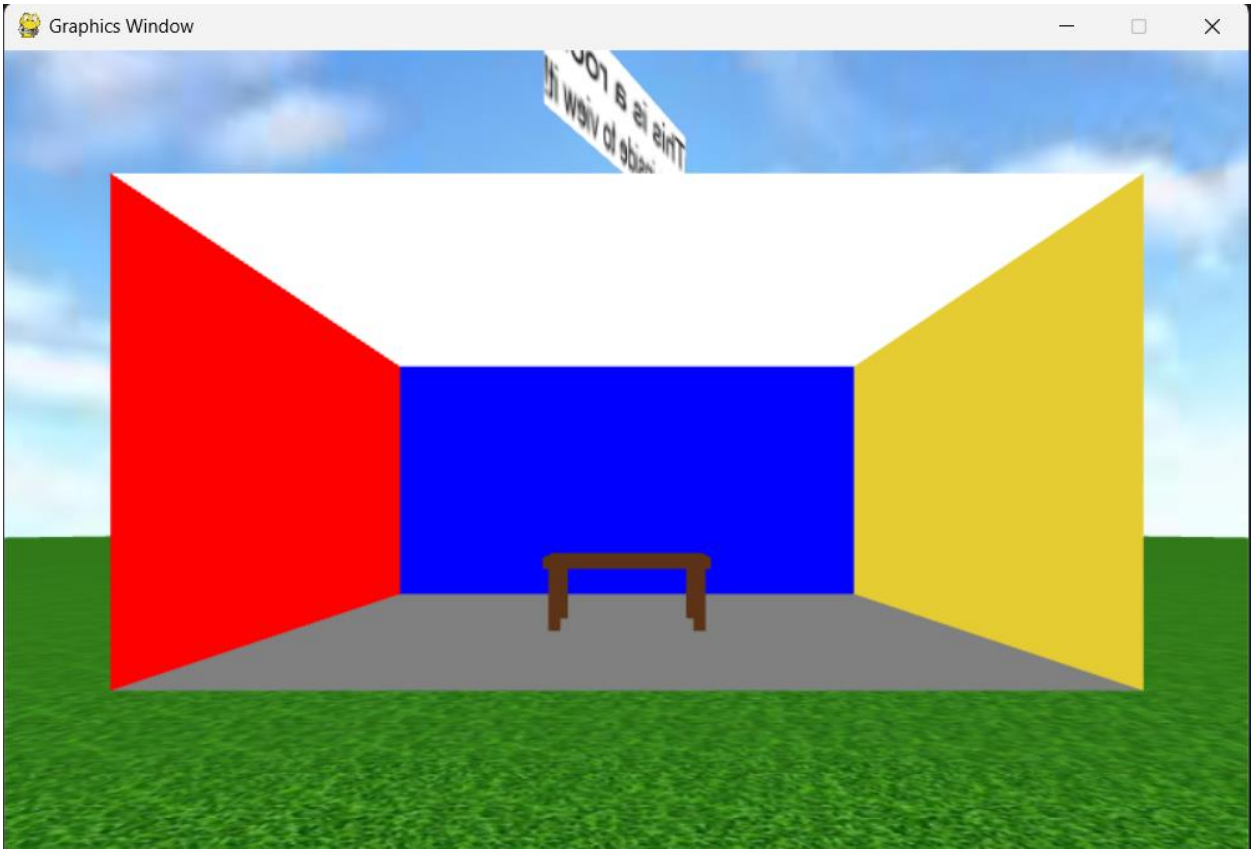
Task 10 – Optional top camera

The main camera erases the sky camera and renders.

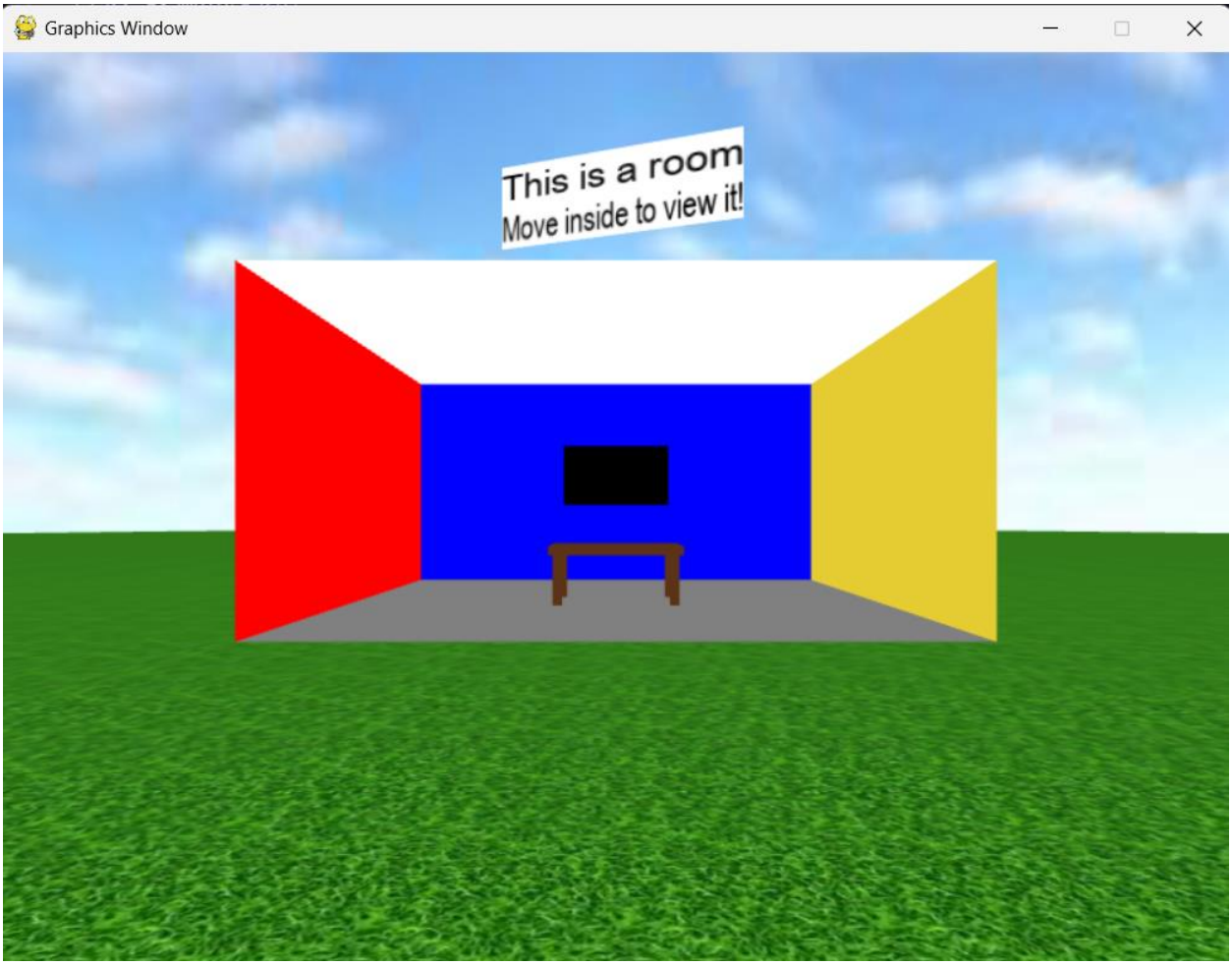
Exercise 4.5 – Furnished room + animated ceiling fan

Task 1 – Keep the outdoor + room setup

Task 2 – Add a table in the center



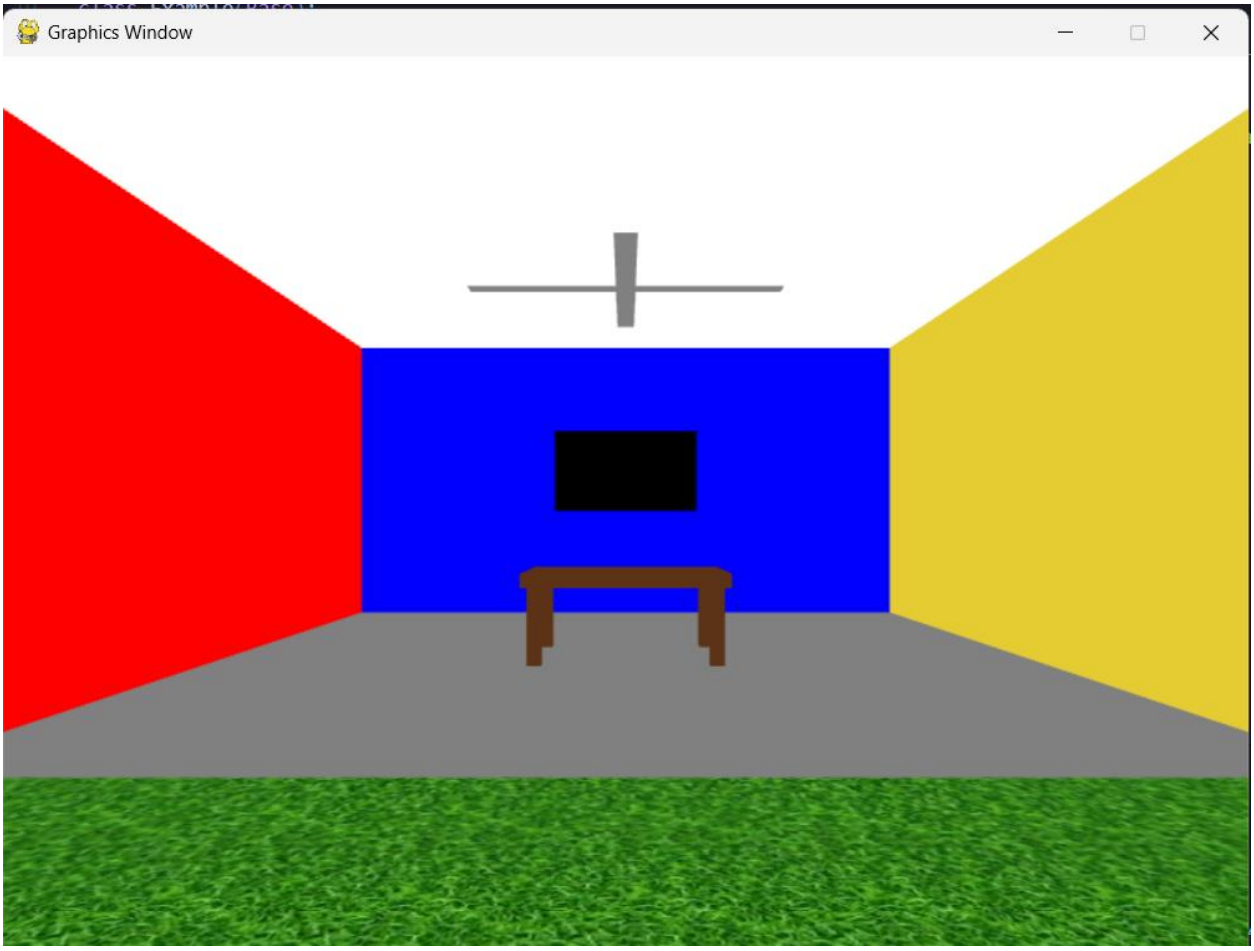
Task 3 – Put a TV on the blue wall (the −Z wall)



Task 4 – Create the ceiling fan hub

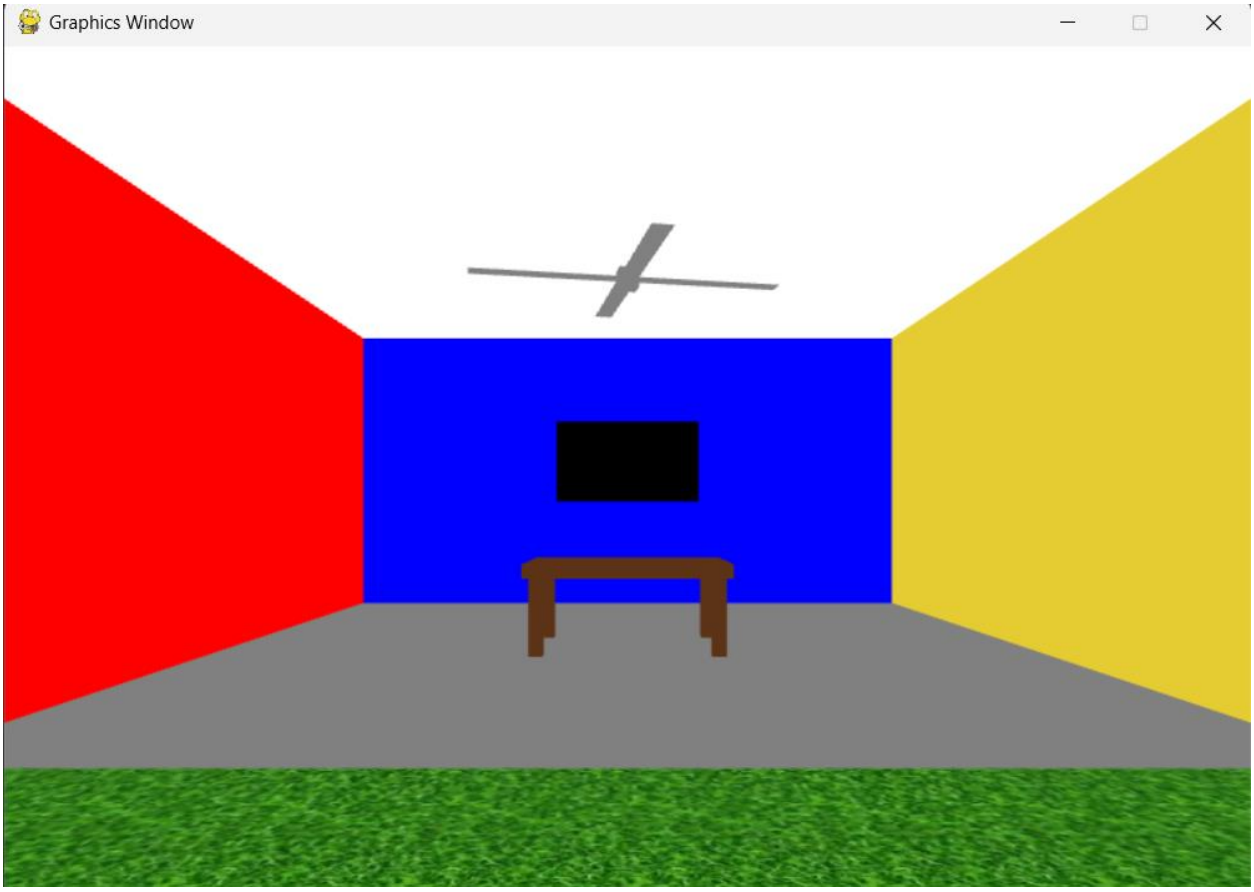
Task 5 – Make 4 pivots around the hub

Task 6 – Attach 1 blade to each pivot



Task 7 – Animate (the key part)

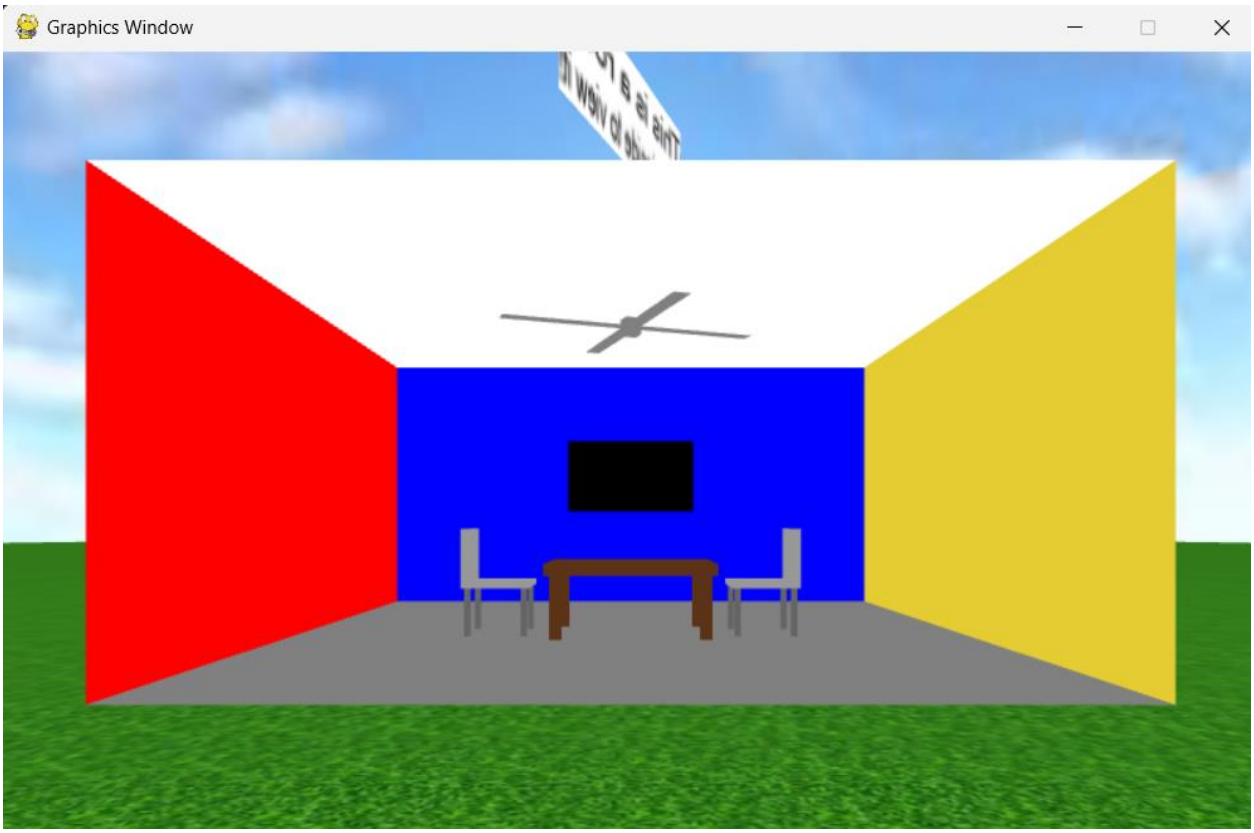
```
154     def update(self):
155         for t in self.text_objects:
156             t.rotate_y(0.3 * self.delta_time)
157         self.rig.update(self.input, self.delta_time)
158         self.fan_hub.rotate_y(4.0 * self.delta_time)
159         self.renderer.render(self.scene, self.sky_camera)
160         self.renderer.render(self.scene, self.camera)
```



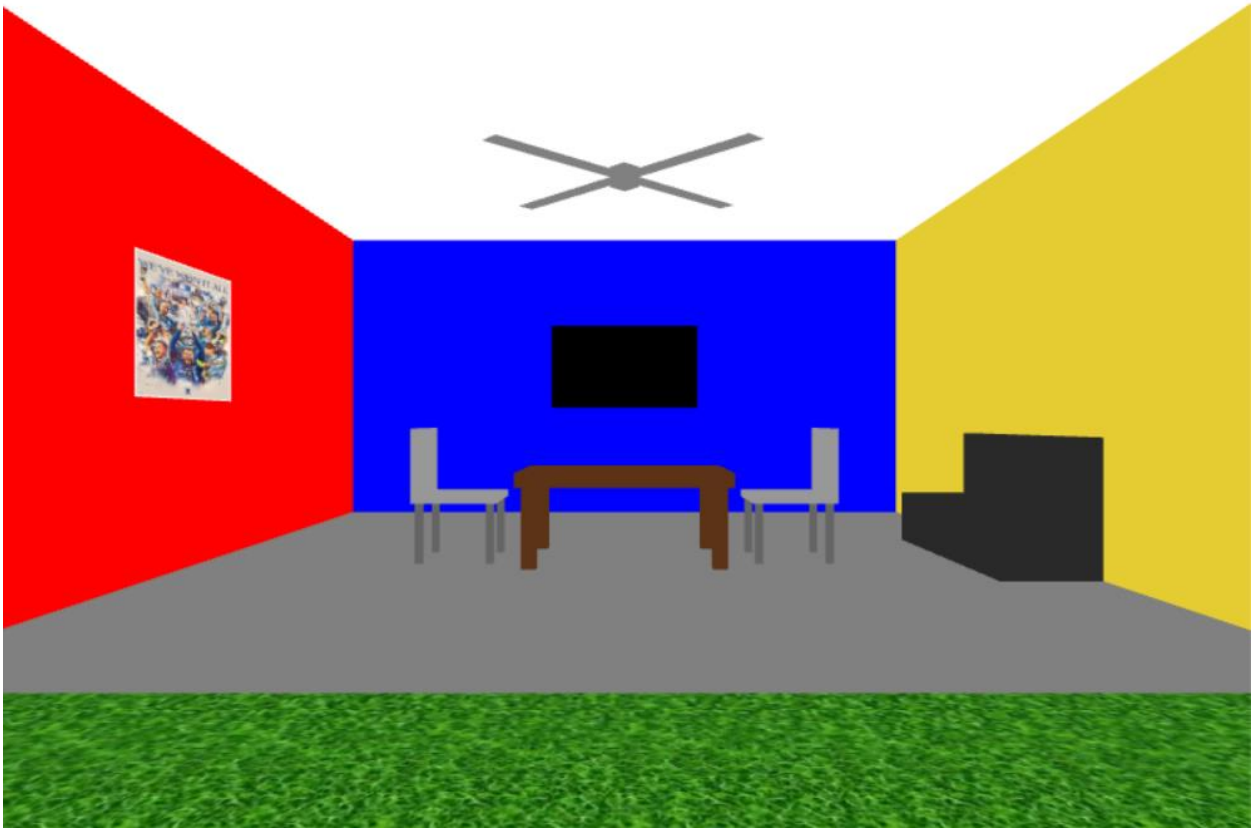
Exercise 4.6 – Furnish & decorate the room (chairs, sofa, picture, double-sided window)

Task 1 – Reuse 4.5 scene

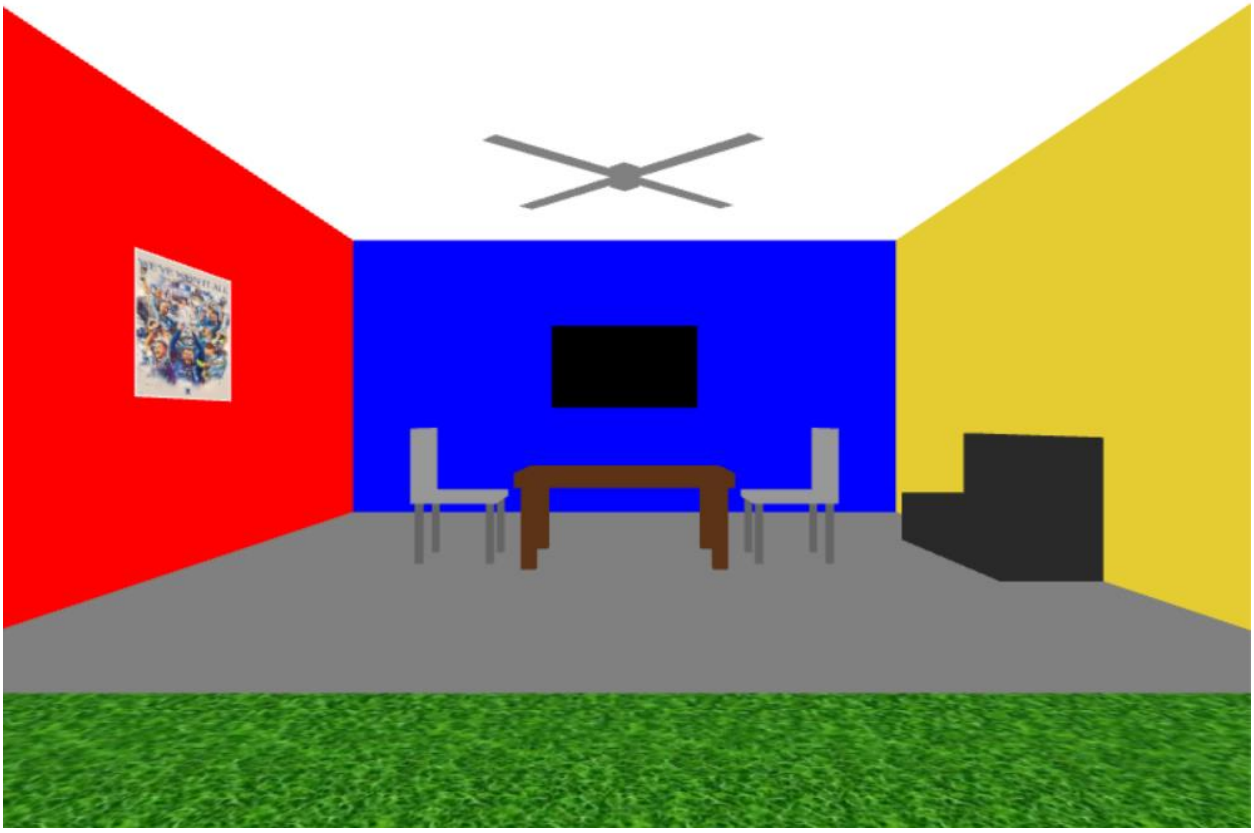
Task 2 – Add two chairs around the table



Task 3 – Add a sofa along the right (yellow) wall



Task 4 – Add a wall picture on the left (red) wall



Task 5 – Build the 2×2 decorative window (5 rectangles) on the front (green) wall

Task 6 – Explain (for report)

1. Why did we need two windows (outside + inside)?

Ans: From my perspective, the outside window is for looking from outside the room, and the inside window is for looking from inside the room.

2. Why do we add a backing rectangle?

Ans: The backing rectangle hide the spaces between the glasses, helping the window looks the same color as the wall → gaps will look green, not black.

3. Why are the picture and TV placed at ... + 0.05 from the wall?

Ans: Because it can stick inside the wall, or glitching. So we might not see the picture and the TV.

Exercise 4.7 – Decorate the Outside (Fence, Gate, Garden)

Task 1 – Read the current room bounds

```
25        print("Initializing program...")
26        room_w, room_h, room_d = 6.0, 3.0, 6.0
```

```
87        back.set_position([0, room_h / 2, -room_d / 2])
```

```
94        front.set_position([0, room_h / 2, room_d / 2])
```

Task 2 – Choose a fence margin

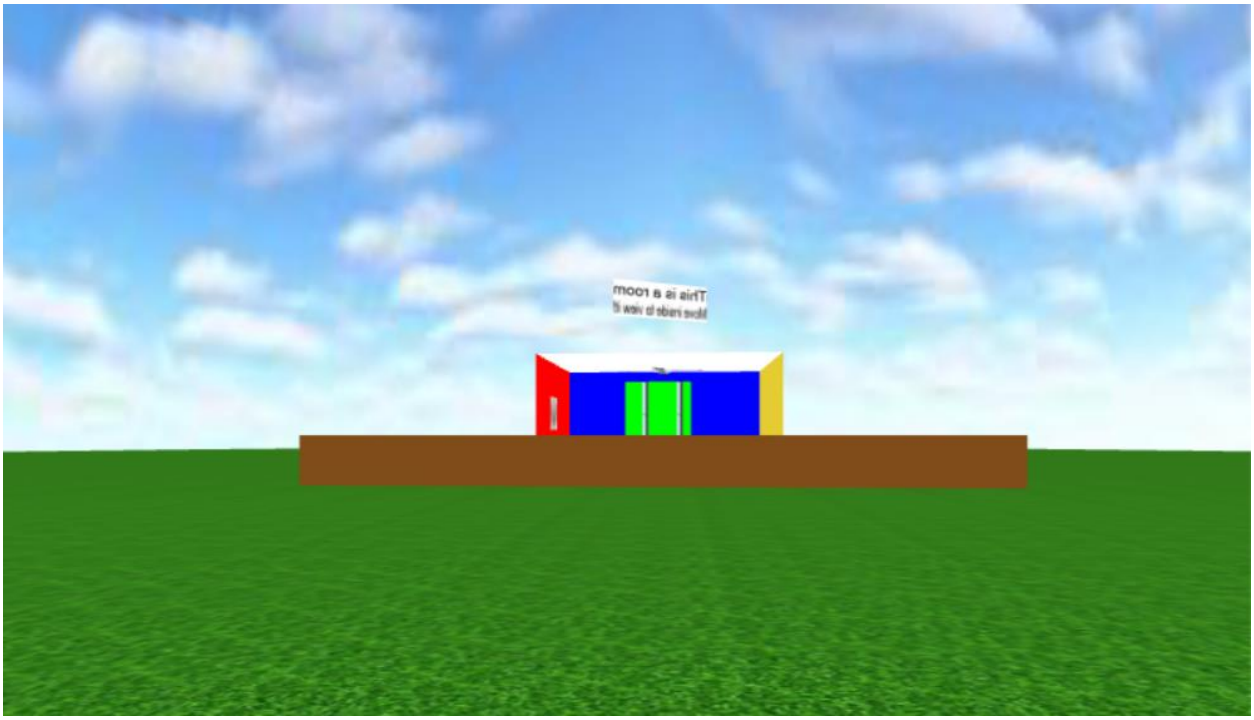
$$\text{fence\_x} = \frac{\text{room\_w}}{2} + \text{margin}$$

$$\text{fence\_z} = \frac{\text{room\_d}}{2} + \text{margin}$$

➔ fence_x = 6/2 + 4 = 7

➔ fence_x = 6/2 + 4 = 7

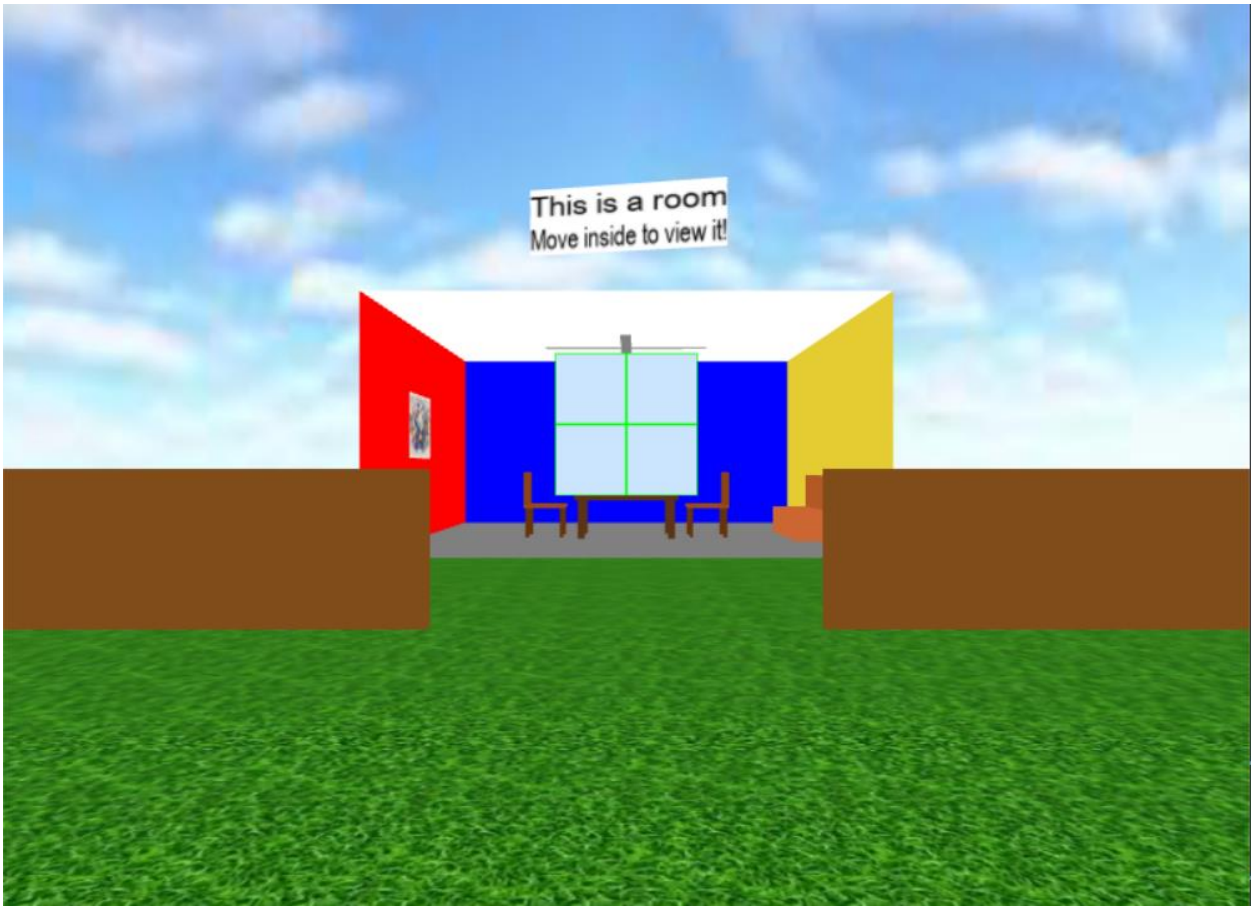
Task 3 – Build 4 axis-aligned fence segments



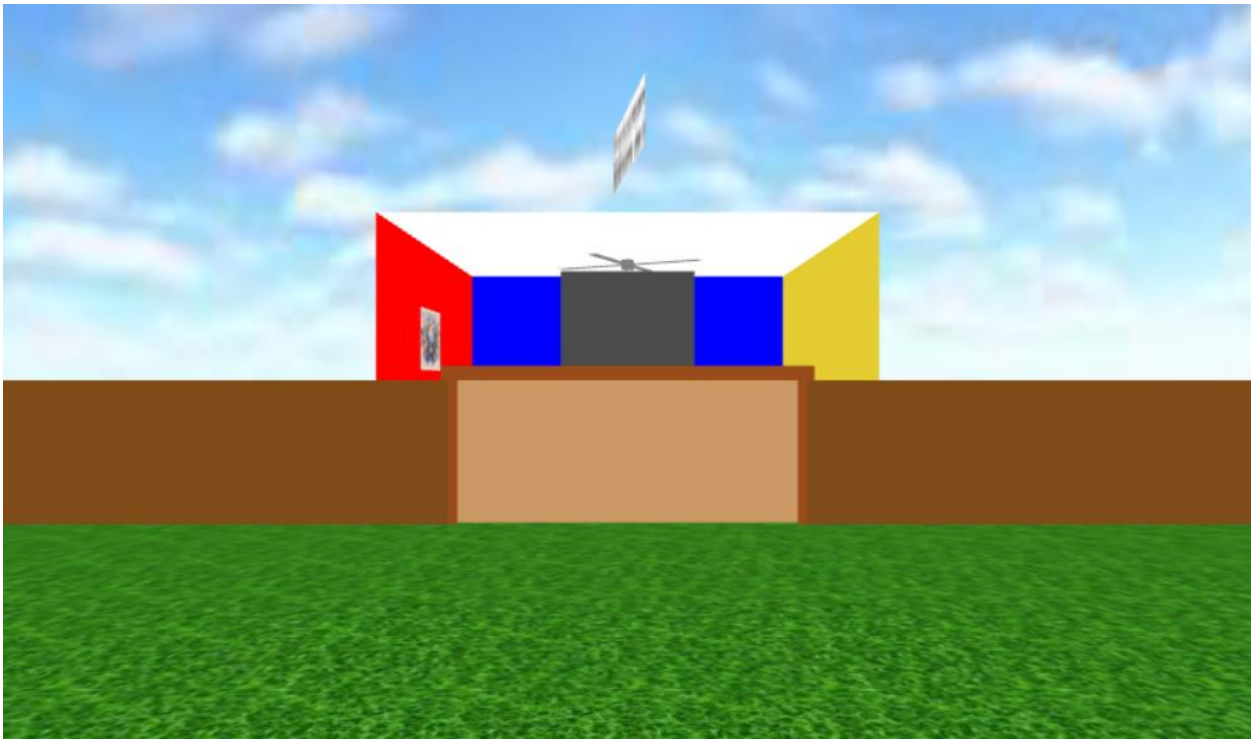
Why do we not rotate these fences?

Ans: If we rotate, these fences might cut across the room or be tilted to one side. So we keep axis-aligned to ensure that no accidental crossing through the room.

Task 4 – Make an opening for the gate



Task 5 – Build the gate from boxes



Task 6 – Add garden objects

Task 7 – Short written part

1. Why do we compute fence_x and fence_z from the room size instead of hard-coding numbers?

Ans: Because the fence will automatically adjust if the room size changes. Keeps the code flexible, reusable, and avoids manually updating values.

2. Why do we place TV/picture/window at ... + 0.05 from the wall?

Ans: To prevent z-fighting, which occurs when two surfaces are exactly at the same position. By creating a small gap so objects render correctly and look visually separated from the wall.

3. What would happen if we rotated the fence segments instead of keeping them axis-aligned?

Ans: The fence could intersect the room incorrectly or be misaligned. Also it would make placing the gate and other objects harder.